最小安全距离计算算子 TreeLineMinDistance

自定义算子工程

通过率： **76%** 通过人数： **34** 尝试人数： **45** 开始答题 提交开源仓

题目内容 提交排名

题目内容

描述

CANNJudge hansen277 退出

TreeLineMinDistance 是面向输电通道智能巡检场景的自定义计算算子。算子接收一组树障三维点和一组输电导线三维线段，对每个树障点计算其到当前批次中**全部导线线段的最小欧氏距离**，并返回距离最近的线段索引。

对于批次 b 中的树障点 P 和线段 j（起点 A、终点 C），点到**有限线段**的最小距离定义为：

```
AC = C - A,  AP = P - A,  L = AC · AC

若 L > 0（普通线段）：
    t  = (AP · AC) / L
    t' = min(1, max(0, t))        # 投影系数截断到 [0, 1]
    Q  = A + t' · AC              # 线段上的最近点
    d(P, S_j) = ||P - Q||_2

若 L = 0（零长度线段，A 与 C 完全相等）：
    d(P, S_j) = ||P - A||_2       # 线段退化为三维点
```

- 当 `0 ≤ t ≤ 1` 时，最近点在线段内部；当 `t < 0` 时最近点为起点 A；当 `t > 1` 时最近点为终点 C。本题计算的是点到**有限线段**的距离，不是点到无限延长直线的距离。
- 对每个树障点在线段维做归约：

```
min_distance[b, i]        = min_{0 <= j < M} d(P[b,i], S[b,j])
nearest_segment_idx[b,i]  = argmin_{0 <= j < M} d(P[b,i], S[b,j])
```

- **平局规则**：当多条线段与树障点的距离完全相同时，必须返回其中**索引最小**的线段。例如 d(P, S2) = d(P, S5) = 1.5 时， `nearest_segment_idx` 必须输出 2。
- 当树障点位于某条线段上时，最小距离为 0，索引同样按最小索引优先规则确定。

算子不负责树木识别、导线识别、点云分割、导线拟合及风险等级判定，只完成上述距离计算与最近线段索引归约。

输入输出规格

张量	方向	形状	数据类型	含义
tree_points	输入	[B, N, 3]	float32 / float16	树障点三维坐标 (x, y, z)
line_segments	输入	[B, M, 2, 3]	float32 / float16	导线线段， `[b,m,0,:]` 为起点、 `[b,m,1,:]` 为终点
min_distance	输出	[B, N]	与输入相同	每个树障点到全部线段的最小欧氏距离
nearest_segment_idx	输出	[B, N]	int32	最近线段索引，取值范围 [0, M-1]

维度含义与范围：

- B：批次大小，B ∈ [1, 8]；N：每批树障点数，N ∈ [1, 16384]；M：每批线段数，M ∈ [1, 1024]。B、N、M 均必须大于 0，不支持零元素张量。
- 坐标最后一维固定为 3，对应 x、y、z。
- 两个输入的批次维 B 必须一致，**不支持广播**。
- 两个输入必须使用相同数据类型；仅支持 float16 和 float32。
- 输入坐标建议范围 [-1000.0, 1000.0]，必须为有限实数，不支持 NaN、+Inf、-Inf。
- 输入输出均为连续存储的 ND 张量，不要求支持非连续 Tensor、负 stride、转置视图或任意 stride 布局。
- N 和 M **均可能不是 32 的整数倍**（典型值：N = 3、19、33、127、1003、4097；M = 7、19、31、33、129、513），实现不得假定任何维度或分块满足 32 字节对齐，尾块无效数据不得影响结果。

属性参数

无属性参数（attr_desc 为空）。

功能示例

示例 1：投影落在线段端点外侧

```
B=1, N=3, M=1
tree_points   = [(-3,4,0), (5,0,0), (13,-4,0)]
line_segments = [((0,0,0), (10,0,0))]

P0=(-3,4,0)：t=-0.3 < 0，最近点为起点 (0,0,0)，d = 5
P1=(5,0,0) ：t=0.5，最近点 (5,0,0) 在线段上，d = 0
P2=(13,-4,0)：t=1.3 > 1，最近点为终点 (10,0,0)，d = 5

min_distance        = [5.0, 0.0, 5.0]
nearest_segment_idx = [0, 0, 0]
```

示例 2：两条线段距离相同，返回最小索引

```
B=1, N=1, M=2，P = (0,0,0)
S0 = ((1,1,0), (2,1,0))   -> 最近点 (1,1,0)，d = sqrt(2) ≈ 1.41421
S1 = ((1,-1,0), (2,-1,0)) -> 最近点 (1,-1,0)，d = sqrt(2) ≈ 1.41421

min_distance        = [1.41421]
nearest_segment_idx = [0]      # 距离相同，取索引 0 而非 1
```

示例 3：零长度线段

```
B=1, N=1, M=2，P = (0,0,0)
S0 = ((3,4,0), (3,4,0))   # 零长度线段（退化为点），d = 5
S1 = ((0,6,0), (0,7,0))   # t < 0，最近点 (0,6,0)，d = 6

min_distance        = [5.0]
nearest_segment_idx = [0]
```

精度要求

```
|actual - expected| <= atol + rtol * |expected|
```

数据类型	rtol	atol	索引要求
float32	1e-4	1e-4	完全一致
float16	1e-3	1e-3	完全一致

- 距离结果逐元素全部满足误差要求，不允许只比较平均误差。
- `nearest_segment_idx` 为 int32，必须逐元素精确匹配。

- 相同距离测试使用严格对称的几何数据；普通测试会避免第一小与第二小距离过于接近。

任务要求

1. **功能正确性**：正确处理单批次/多批次、单点/单线段、大规模点线组合、投影在线段内部或端点、投影落在线段两侧、零长度线段、多条线段距离相同、float16/float32 以及非 32 整倍数尾块等全部场景。

2. **float16 中间计算**：输入为 float16 时，坐标差值、向量点积、平方和、除法、投影系数、Clamp、距离比较、Min/ArgMin 及平方根计算均必须使用 float32（或等效高精度）完成，最终距离再转回 float16 输出；最近线段索引也必须基于高精度中间距离确定。

3. **零长度线段**：不得执行除以 L = 0 的运算，不得产生除零、NaN 或 Inf；是否为零长度以量化后的实际值为准，不使用模糊阈值。

4. **输出语义**：可以先比较距离平方完成 Min/ArgMin（argmin d 等价于 argmin d^2），但最终 `min_distance` 必须输出实际欧氏距离，不得输出距离平方；平方根次数应尽量少。

5. **内存与越界**：不得发生输入输出越界读写；尾块/填充数据不得产生虚假的最小距离。**不建议**生成完整的 (B, N, M) 或 (B, N, M, 3) 距离矩阵后再统一归约，该方式会带来过高的 Global Memory 与 UB 压力。

6. **性能优化**：在保证正确性和精度的前提下，充分利用昇腾 NPU 硬件特性，包括向量化计算、合理 Tiling、多核并行、Global Memory 访问优化、片上数据复用、减少重复搬运、减少平方根次数、优化 ReduceMin/ArgMin 以及搬入—计算—搬出流水，使算子性能更优。

我的提交记录

暂无提交记录